

Chapter 9. Gate A-G가 Book-forge에서 실제로 측정하는 것

이 챕터에서 배우는 것

- 7개 Gate(A-G) 중 Book-forge가 실제로 값을 채우는 것은 몇 개인지
- 왜 어떤 Gate는 자주 "N/A"로 나오는지
- 한 챕터를 실제로 채점해보면 어떤 숫자가 나오는지

이런 분이 먼저 읽으면 좋습니다: "58개 지표, 7개 Gate"라는 Agent-Evaluator SDK의 전체 규모에 압도됐지만, 실제 프로젝트 하나가 그중 정확히 무엇을 쓰는지 궁금한 분.

9.1 33개 Config·25개 트래커가 아니라, Book-forge가 실제로 쓰는 것만

Agent-Evaluator SDK 전체는 25개 네이티브 트래커와 33개 Harness Config를 제공한다. 이 챕터는 그 전체 카탈로그를 나열하지 않는다 — 8장에서 확인한 Book-forge의 실제 에이전트 6개가 **실제로 건드리는** Gate만 정리한다.

Gate	Book-forge가 채우는 값	어느 에이전트가 기여하는가
A 목표 달성	TaskCompletionTracker + AccuracyEvaluator 블렌딩 ($0.6 \times \text{TCR} + 0.4 \times \text{Accuracy}$)	전체 — <code>ground_truth</code> 를 넘기는 모든 에이전트
B 행동 무결성	LoopDetectionConfig 기반 반복 탐지	ReviseAgent(<code>review_loop.py</code>)만 — 나머지 에이전트는 대부분 N/A
C 신뢰성	HallucinationDetector(RAG 근거 대조)	ChapterDrafterAgent·ChatAgent(<code>rag_mode=True</code>)
D 성능 계약	LatencyTracker(항상) + SLAConfig 위반 여부	전체(LatencyTracker는 항상 기록)

Gate	Book-forge가 채우는 값	어느 에이전트가 기여하는가
E 보안 경계	5개 보안 트래커 + ThreatSeverityConfig	ChapterDrafterAgent가 명시적, 나머지는 <code>enable_security_metrics=True</code> 로 상시 on
F 다중 에이전트	AgentCoordinationTracker + ConflictResolutionConfig	ReviewPanel(5장)만 — 유일하게 값이 나오는 지점
G 관측성	ExplainabilityConfig(PlannerAgent)	PlannerAgent만 명시적, 나머지는 대부분 N/A

9.2 "N/A"가 버그가 아니라 정직한 신호인 이유

이 표를 처음 보면 Gate B·F·G가 유독 자주 비어 보인다는 인상을 받을 수 있다. 이건 계측 누락이 아니라 **그 Gate가 실제로 측정할 신호 자체가 그 태스크에 없기 때문**이다.

- Gate F(다중 에이전트)는 `AgentCoordinationTracker` 가 `track_interaction()` 호출을 필요로 한다(5장 § 5.3). PlannerAgent 하나만 도는 태스크에는 애초에 "다른 에이전트와의 상호작용"이라는 사건 자체가 없다 — N/A가 정직한 답이다.
- Gate B(행동 무결성)의 루프 탐지도 마찬가지다 — 한 번만 호출되는 함수에는 "반복"이라는 개념이 성립하지 않는다.

`monitor.py` (agent-evaluator SDK)의 원칙이 정확히 이렇다 — 데이터가 없는 Gate는 억지로 기본값(예: 1.0이나 0.5)을 채우지 않고 N/A로 남긴다. **거짓으로 만점을 주는 것보다 "측정하지 않았다"고 정직하게 말하는 편이, 이후 그 Gate 점수를 신뢰할지 판단하는 사람에게 더 유용하다.**

9.3 Gate A의 실제 계산 — TCR과 Accuracy를 섞는다

Gate A는 이 책이 가장 자주 언급하는 Gate다 (PlannerAgent·TOCDesignerAgent·ChapterDrafterAgent 전부 기여). 실제 점수 계산은 다음 공식을 따른다(`CLAUDE.md`에 문서화된 Book-forge 전역 설정).

```
_a_score = gate_a_tcr_weight × TCR컴포넌트 + (1 - gate_a_tcr_weight) × 나머지 평균
```

기본값 `gate_a_tcr_weight=0.4` — 태스크 완료율(TCR)이 40%, 나머지 (AccuracyEvaluator 블렌딩·ResponseQualityEvaluator 등)가 60%를 차지한다. 14장에서 이 가중치를 프로젝트마다 다르게 조정하는 실제 코드(`.env`)를 다룬다.

9.4 실제 채점 사례 — 한 챕터의 Gate 점수

이 책을 준비하며 실제로 `book-forge new --source ...` 로 챕터 하나를 생성했을 때 나온 실제 로그다(이 세션의 실제 실행 기록).

```
📄 이 초안의 Gate 점수 (참고용 — 전체 판정은 book-forge gate로):  
⚠️ A (목표 달성): 0.663  
  · B (행동 무결성): N/A  
✅ C (신뢰성): 0.708  
❌ D (성능 계약): 0.135  
✅ E (보안 경계): 1.000  
  · F (다중 에이전트): N/A  
⚠️ G (관측성): 0.694
```

이 예시가 § 9.2의 주장을 그대로 뒷받침한다 — B·F는 N/A(그 태스크에 반복이나 다중 에이전트 상호작용이 없었으므로), D는 0.135로 낮게 나왔다(로컬 35B 모델이 SLA 60초 기준을 자주 초과했기 때문 — 로컬 추론의 실제 트레이드오프다). E는 1.000인데, 이는 "보안이 완벽하다"는 뜻이라기보다 이 특정 태스크에서 5개 보안 트래커가 위반을 하나도 못 찾았다는 뜻이다 — Gate 점수는 항상 **그 태스크에서 실제로 검사한 것에 한해서만** 유효하다.

⚠️ **N/A와 낮은 점수를 혼동하지 말 것:** `N/A` 는 "측정 대상이 아니었다"이고, `0.135` (D)는 "측정했고, 기준을 못 넘었다"다. 이 둘을 같은 것으로 취급해 평균에 섞으면 안 된다 — 실제로 `HarnessEvaluationGate` 는 데이터 없는 Gate를 평균에서 제외한다.

이 챕터의 핵심

- **Book-forge는 7개 Gate 중 정확히 필요한 신호만 채운다.** 나머지는 억지로 채우지 않고 N/A로 남긴다.

- **N/A는 결함이 아니라 정직함이다.** "이 태스크에는 이 Gate가 측정할 신호 자체가 없었다"는 뜻이다.
- **Gate A는 TCR과 Accuracy를 가중 평균한다(기본 0.4:0.6).** 이 가중치는 조정 가능하다(14장).
- **실측 사례에서 Gate D(성능)가 자주 낮게 나온다.** 로컬 모델의 지연 시간이 SLA 기준(60초)을 넘기는 경우가 실제로 있다.

참고 자료

- `Agent-Evaluator/CLAUDE.md` — Gate별 트래커 기여 방식 전체 표
 - `src/book_forge/eval/monitor.py` — `build_book_monitor()`
 - `src/book_forge/cli/commands/draft_cmd.py` — 챗터 생성 직후 Gate 점수를 출력하는 코드
-

다음 챗터는 Gate A-G가 손대지 않는 영역 — LLM을 아예 호출하지 않는 순수 정적 검증기 3종을 다룬다.